

Angewandte Wahrscheinlichkeitsrechnung und Statistik

Einführung in R und RStudio

Marcus Wurzer

Übersicht

- ➊ R - Allgemeine Informationen
- ➋ RStudio
- ➌ Arbeiten mit R
- ➍ Dokumentation
- ➎ Sonstiges

Abschnitt 1

R - Allgemeine Informationen

Über R

- R ist ein frei verfügbares (General Public Licence - GPL) Programm für statistisches Rechnen und statistische Grafiken
- R ist plattformunabhängig - Versionen für Windows, Linux und Mac (OS X) sind verfügbar
- Geschätzte Anzahl der Personen, die regelmäßig mit R arbeiten: mehrere Millionen
- Firmen wie Google, Pfizer, Bank of America, die InterContinental Hotels Group oder Shell verwenden R für ihre statistischen Analysen

- R ist eigentlich eine komplette Programmiersprache - es implementiert einen Dialekt der statistischen Programmiersprache S
- R enthält eine große Anzahl von vorgefertigten Mechanismen, um Daten zu organisieren sowie Berechnungen mit diesen durchzuführen und sie grafisch darzustellen

The R Project for Statistical Computing

- Zentrale R-Anlaufstelle: <http://www.r-project.org>
- Die R-Homepage enthält neben den Downloadlinks auch noch Informationen über das **R Project** und die **R Foundation**, sowie eine **Documentation**-Sektion und **Links** zu Projekten, die mit R in Zusammenhang stehen


[\[Home\]](#)

Download

[CRAN](#)

R Project

[About R](#)
[Logo](#)
[Contributors](#)
[What's New?](#)
[Reporting Bugs](#)
[Conferences](#)
[Search](#)
[Get Involved: Mailing Lists](#)
[Get Involved: Contributing](#)
[Developer Pages](#)
[R Blog](#)

R Foundation

[Foundation](#)
[Board](#)
[Members](#)
[Donors](#)
[Donate](#)

Help With R

[Getting Help](#)

Documentation

[Manuals](#)
[FAQs](#)
[The R Journal](#)
[Books](#)
[Certification](#)
[Other](#)

Links

[Bioconductor](#)
[R-Forge](#)
[R-Hub](#)
[GSoc](#)

The R Project for Statistical Computing

Getting Started

R is a free software environment for statistical computing and graphics. It compiles and runs on a wide variety of UNIX platforms, Windows and MacOS. To [download R](#), please choose your preferred [CRAN mirror](#).

If you have questions about R like how to download and install the software, or what the license terms are, please read our [answers to frequently asked questions](#) before you send an email.

News

- **R version 4.3.1 (Beagle Scouts)** has been released on 2023-06-16.
- R/Basel, a *useR!* Regional 2023 event, has a call for abstracts open until Friday 26 May 2023.
- **R version 4.2.3 (Shortstop Beagle)** has been released on 2023-03-15.
- You can support the R Foundation with a renewable subscription as a [supporting member](#)

News via Twitter

[News from the R Foundation](#)

Other Social Media

Follow the R Foundation on [Mastodon](#) or [LinkedIn](#).

CRAN

- CRAN steht für **C**omprehensive **R** **A**rchive **N**etwork
- CRAN ist ein Server-Netzwerk, das die Basisdistribution und die R-Zusatzpakete hostet
- Zentraler Server: <http://cran.r-project.org>
- Neue R-Versionen erscheinen üblicherweise im Abstand von ein paar Wochen

Contributed extension packages

- Zusatzpakete (“Contributed extension packages”) sind Erweiterungen, die zusätzliche Funktionalität zur Verfügung stellen
- Die Packages werden nach dem Download lokal gespeichert, unter Windows z. B. im Ordner `...\R\R-4.3.1\library`
- Die R-Basisinstallation bringt bereits eine Reihe von wichtigen Packages mit
- Fast 20.000 weitere Packages (Stand: 2023-07) können von CRAN heruntergeladen werden, momentan kommen pro Tag im Schnitt mehrere neue dazu

- Diese decken verschiedenste Methoden und Thematiken ab, z. B. die Analyse von Sprachmustern, die Analyse des menschlichen Genoms, das Berechnen von Klimaveränderungen, die Analyse von Derivaten zur Entwicklung von Finanzdienstleistungen usw.
- Packages können entweder über CRAN oder über ein lokales .zip-File installiert werden
- Nachdem ein Package erfolgreich installiert wurde, muss es noch geladen werden, um die enthaltenen Funktionen, Datensätze etc. nutzen zu können

Abschnitt 2

RStudio

Editoren und IDEs

- Wenn R heruntergeladen und installiert wurde und man das Programm ausführt, öffnet sich ein Fenster mit der R-Konsole (unter Windows und Mac OS-X) oder ein Terminal (unter Linux), und man kann damit beginnen, R-Code einzugeben und ausführen zu lassen
- Sinnvoll ist aber die Verwendung von **Editoren** und/oder **IDEs** (**I**ntegrated **D**evelopment **E**nvironments), die ein komfortableres Arbeiten ermöglichen
- Editoren sind eigenständige Anwendungen, die sich mit einer installierten R-Version verbinden lassen und für das Editieren von R-Source Code verwendet werden

- IDEs integrieren diverse Entwicklungswerkzeuge (Editoren, Compiler, Debugger etc.) in einem einzigen Programm - der Nutzer muss sich nicht um die Verbindung der einzelnen Bestandteile kümmern
- Empfehlenswert ist der Einsatz von **RStudio**, einer plattformübergreifend verfügbaren Entwicklungsumgebung, die für Einzelnutzer als Open Source Edition zur Verfügung steht
- RStudio kann den einfachen Editor, der in der Basisversion von R integriert ist, komplett ersetzen. R muss nicht mehr gesondert gestartet werden

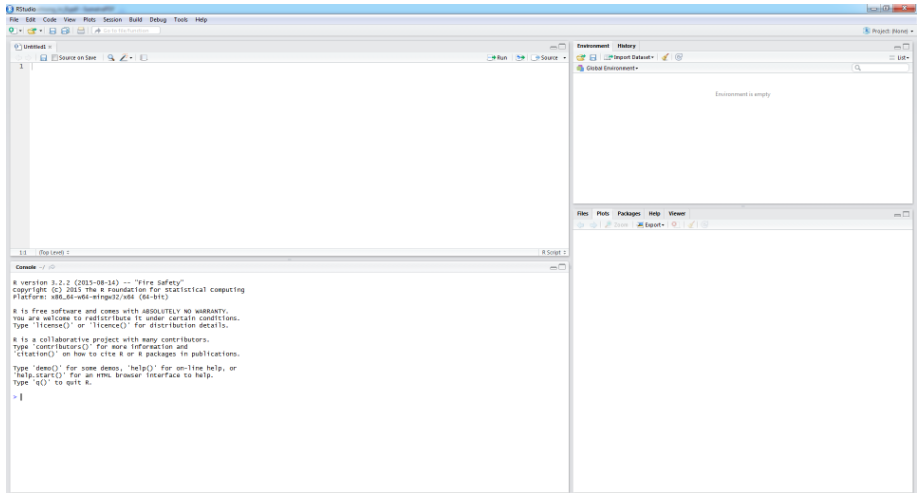
Download und Installation

- RStudio ersetzt R nicht, notwendig ist also eine bereits installierte R-Version!
- RStudio setzt automatisch auf der aktuellsten installierten R-Version auf. Über *Tools* → *Global Options* kann dies aber auch geändert werden
- Download unter:
<https://www.rstudio.com/products/rstudio/download/#download>

Aufbau

- Das Hauptfenster von RStudio umfasst fünf Teile: ein Menü und vier Fenster ('Panels')
- Über das **drop-down menu** können RStudio und R kontrolliert werden
- Pane 1 (oben links) - **Files and Data**: Bearbeiten von R-Code und Anzeige von Datensätzen
- Pane 2 (oben rechts) - **Environment, History, Connections**:
 - ▶ **Environment**: Daten importieren, Objekte im Workspace auflisten
 - ▶ **History**, um den kompletten Code anzuzeigen, der bisher in der Konsole eingetippt wurde

- Pane 3 (unten rechts) - **Files, Plots, Packages, Help, Viewer:**
 - ▶ **Files**, um Dateien zu verwalten
 - ★ **Plots**, um Grafiken anzuzeigen und zu exportieren
 - ★ **Packages** für die Verwaltung der Zusatzpakete
 - ★ **Help**, um Hilfe zu R-Funktionen und -Datensätzen zu bekommen
- Pane 4 (unten links) - **Console, Terminal, R Markdown, Jobs:**
Ausführen von R-Code, Generieren dynamischer Dokumente etc.
- Das Layout der Panes (sowohl Anordnung wie auch Inhalte) kann im Optionsmenü geändert werden



Ausführen von Code

- Tastenkürzel, um Befehle in der Konsole auszuführen:

Beschreibung	Tastenkürzel
Aktuelle Zeile/Auswahl ausführen	Strg + Enter
Gesamten Code ausführen	Strg + Alt + R
Alles vom Anfang bis zur aktuellen Zeile ausführen	Strg + Alt + B

Abschnitt 3

Arbeiten mit R

Grundrechenarten

- Gibt man in der R-Konsole neben der Eingabeaufforderung (Symbol: >) einen R-Befehl ein und drückt die Eingabetaste, so wird der Befehl von R ausgewertet, z. B.:

```
1  
## [1] 1
```

- Auf der folgenden Folie finden sich einige Beispiele für die Grundrechenarten

```
3 + 3          #Addition
## [1] 6

3 - 3          #Subtraktion
## [1] 0

3 * 3          #Multiplikation
## [1] 9

3 / 3          #Division
## [1] 1

3 ^ 5          #Potenzieren
## [1] 243

sqrt(81)       #Quadratwurzel
## [1] 9

243 ^ (1 / 5)  #a-te Wurzel
## [1] 3
```

- Leerzeichen sollten aus Gründen der Übersichtlichkeit verwendet werden, müssen es aber nicht. $1+1$ funktioniert ebenso wie $1 + 1$

Dezimaltrenner und Gruppierungszeichen

- Gruppierungszeichen werden in R nicht verwendet, die Zahl Tausend wird also nicht als 1.000 angeschrieben, sondern als 1000
- Zahlen mit Nachkommastellen werden folgendermaßen angeschrieben: 27.28. Der Dezimaltrenner ist also nicht wie im Deutschen der Beistrich (**Kommazahlen**, Nach**kommastellen**), sondern der Punkt:

```
1,5 + 1,5 # falsch
## Error: <text>:1:2: Unerwartete(s) ',',
## 1: 1,
##      ^
```

```
1.5 + 1.5 # richtig
## [1] 3
```

Objekte

- R ist eine objektorientierte Programmiersprache
- Alle Daten und Funktionen (wie `*`, `+`, `sin()`, `max()` usw.) in R sind **Objekte**
- Objekte, die in R erzeugt werden, können mit Hilfe von Zuweisungen (`<-`) “gespeichert” und später weiterverwendet werden

```
x <- 3 #Zuweisung des Werts 3 zum Objekt x
x
## [1] 3
y <- 5
x + y
## [1] 8
z <- x + y
z
## [1] 8
(aa <- 3) #Zuweisung und Ausgabe von aa
## [1] 3
```

Skripte

- Sobald man mehrere Kommandos hintereinander ausführen will, empfiehlt es sich, diese zu einem Skript zu kombinieren. Dies geschieht im **Files and Data** Pane
- Mit der Tastenkombination Strg + R können einzelne Zeilen oder markierte Teile des Codes an den Interpreter geschickt werden
- Skripte lassen sich laden und speichern. R-Skripte haben üblicherweise die Endung .R

Der Workspace

- Die erzeugten Objekte werden im **Workspace** (Arbeitsplatz) abgelegt
- Der Befehl `ls()` zeigt alle Objekte an, die sich derzeit im Workspace befinden:

```
ls()  
## [1] "aa"           "hook_output" "x"           "y"  
## [5] "z"
```

- Der Workspace (mitsamt den Objekten) kann mit dem Befehl `save.image()` für eine spätere Verwendung gespeichert werden (Dateiendung: `.Rdata`).
- Beim Beenden von RStudio wird man außerdem gefragt, ob man den Workspace sichern möchte
- Üblicherweise ist das Speichern nicht notwendig

Das Arbeitsverzeichnis

- Standardmäßig werden Daten, Workspace etc. im derzeitigen **working directory/Arbeitsverzeichnis** gespeichert
- Das Arbeitsverzeichnis kann über den Befehl `getwd()` abgerufen und mit `setwd()` geändert werden:

```
getwd()  
setwd("C:/") #unter Windows
```

- Hinweis: Beim der Angabe von Pfaden muss man / oder \\ verwenden, \ (wie sonst in Windows üblich) funktioniert nicht
- Wenn man ein R-Skript aus einem Verzeichnis heraus öffnet, wird dieses als Arbeitsverzeichnis gesetzt

Funktionen

- R ist eine funktionale Programmiersprache, alle Funktionen arbeiten nach demselben Schema: Die Funktion wird durch ihren Namen aufgerufen, in runden Klammern werden diverse Funktionsargumente übergeben, die durch Kommas getrennt sind: `Funktionsname(arg1, arg2, ...)`
- Es können natürlich auch eigene Funktionen geschrieben werden, der große Vorteil von R liegt aber darin, dass es für alle nur denkbaren statistischen Fragestellungen schon vorgefertigte Funktionen gibt

- Manchen Funktionen müssen zwingend ein oder mehrere Argumente übergeben werden (z. B. `sqrt()`), manchen nicht, da es für die Argumente schon Voreinstellungen (default values) gibt (z. B. `ls()`). Diese können natürlich auch überschrieben werden

```
sqrt()  
## Error in sqrt(): 0 Argumente an 'sqrt' übergeben, welches 1 benötigt  
sqrt(25)  
## [1] 5
```

- Funktionen können geschachtelt werden, wobei die Funktionen von innen nach außen abgearbeitet werden:

```
log(sqrt(25))  
## [1] 1.609438  
log(5)  
## [1] 1.609438
```

- Die `args()`-Funktion erlaubt es, sich die einzelnen Argumente einer Funktion anzeigen zu lassen:

```
args(log)
## function (x, base = exp(1))
## NULL
```

- Besser ist es aber, sich direkt die ganze Funktionsdokumentation anzusehen (siehe unten)

Abschnitt 4

Dokumentation

Anleitungen

- Diverse Anleitungen, die wichtige Themen bezüglich der Verwendung von R abdecken, werden bei der Installation von R automatisch in das Unterverzeichnis `...\\doc\\manual` des Ordners entpackt, in dem sich die R-Installation befindet
- Alle Anleitungen lassen sich auch auf der R-Homepage finden
- Außerdem auf der R-Homepage verfügbar: Ein Wiki, eine FAQ-Sektion, eine Liste von Büchern über R etc.

Hilfe zu Funktionen

- `sqrt()` hat also nur ein Argument `x` ohne Voreinstellung, `log` weist zwei Argumente auf, davon hat eines (`base`) die Voreinstellung `exp(1)`, d. h., die Eulersche Zahl e ($\rightarrow$ natürlicher Logarithmus)
- Genauere Informationen zu den Argumenten bekommt man, indem man die Hilfe zu einer Funktion aufruft

- Die Funktion `help()` kann dazu verwendet werden, Hilfe zu einer Funktion zu bekommen. Noch einfacher ist die Verwendung der Abkürzung `?:`

```
help(sqrt)  
?sqrt #dasselbe
```

- Achtung: R unterscheidet Groß- und Kleinschreibung!

```
?Sqrt
```

```
## Keine Dokumentation für 'Sqrt' in angegebenen Paketen  
und Bibliotheken:
```

```
## Sie können '??Sqrt' versuchen
```

- Wenn -wie oben- keine Dokumentation gefunden wird, z. B. weil man den genauen Namen der Funktion nicht kennt, kann man `help.search()` dazu verwenden, die gewünschte Information zu finden (Abkürzung: `??`)

```
help.search("matrix")  
??matrix # dasselbe  
??Matrix # dasselbe! (Fuzzy Matching/Approximate  
           # String Matching)
```

- `help.start()` ruft die Online-Dokumentation von R auf

Hilfe zu Packages

- Im Folgenden ist eine Reihe von Funktionen angegeben, die dazu dienen, Hilfe zu einem bestimmten Package zu erhalten (das Package muss natürlich installiert sein). Wir zeigen dies am Beispiel des `rgl`-Package, welches für die Erstellung von 3D-Grafiken verwendet werden kann:

```
install.packages("rgl")  
library(rgl)
```

```
library() #verfügbare Packages
## Warning in library(): Libraries
##  '/usr/local/lib/R/site-library', '/usr/lib/R/site-library'
## enthalten keine Pakete
search() #geladene Packages
## [1] ".GlobalEnv" "package:rgl"
## [3] "package:knitr" "package:stats"
## [5] "package:graphics" "package:grDevices"
## [7] "package:utils" "package:datasets"
## [9] "package:methods" "Autoloads"
## [11] "package:base"
```

```
help(package = "rgl") #Dokumentation zum Package
                     #rgl
library(help = rgl) #kurze Beschreibung
                   #des Packages
```

```
packageDescription("rgl") #kurze Beschreibung  
                        #des Packages  
ls("package:rgl") #Auflistung der grundlegenden  
                  #Funktionen  
lsf.str("package:rgl") #Auflistung der  
                      #grundlegenden Funktionen  
                      #und Argumente  
vignette(package = "rgl") # Auflistung der  
                          # Vignette(n)  
vignette("rgl", package = "rgl") # Öffnen einer  
                                # Vignette (wenn  
                                # vorhanden)
```

```
RSiteSearch("rgl") #Durchsuchen von  
                  #http://search.r-project.org  
                  #nach dem Schlüsselwort "rgl"  
                  #-> wird im Browser geöffnet  
demo(package = "rgl") #Auflistung der enthaltenen  
                    #Demos (wenn vorhanden)  
demo("abundance", package = "rgl") #Starten der  
                                   #"abundance"-Demo  
data(package = "rgl") #Liste verfügbarer Datensätze  
                     #(wenn vorhanden)
```

Abschnitt 5

Sonstiges

Boolsche Vektoren

- Boolsche Vektoren (TRUE/FALSE → siehe Studienbrief “Datenmanagement in R”) können nicht nur direkt eingegeben werden, sondern sind meist das Ergebnis der Verwendung von logischen Operatoren:

```
ee <- 10:1
ee == 3 # Element von ee weist den Wert 3 auf?
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE FALSE
## [10] FALSE
```

```
ee > 3 # Element ist größer als 3?
## [1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE
## [10] FALSE

ee <= 3 # Element ist kleiner gleich 3?
## [1] FALSE FALSE FALSE FALSE FALSE FALSE FALSE TRUE TRUE
## [10] TRUE

ee > 2 & ee < 8 # Element ist größer als 3 und kleiner als 8?
## [1] FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE FALSE
## [10] FALSE

ee < 2 | ee > 8 # Element ist kleiner als 2 oder größer als 8?
## [1] TRUE TRUE FALSE FALSE FALSE FALSE FALSE FALSE FALSE
## [10] TRUE
```

- Wenn man wissen will, an welcher Stelle sich die Werte befinden, die die Bedingung erfüllen, kann man die Funktion `which()` verwenden

```
which(ee <= 3)
## [1] 8 9 10
```

- `any()` und `all()` informieren darüber, ob mindestens einer oder sogar alle Werte die Bedingung erfüllen

```
any(ee == 3)
## [1] TRUE
all(ee == 3)
## [1] FALSE
```

Stichprobenziehung

- Das Ziehen einer Stichprobe aus einer gegebenen Grundgesamtheit wird mit der Funktion `sample()` durchgeführt
- Da das Ergebnis (pseudo-)zufällig ist, werden bei jeder Ziehung andere Elemente ausgewählt

```
ee
## [1] 10 9 8 7 6 5 4 3 2 1
sample(ee, 3)
## [1] 6 10 2
sample(ee, 3)
## [1] 3 7 8
sample(ee, 3)
## [1] 6 4 2
```

- Die Pseudo-Zufälligkeit erlaubt es uns aber, die Ergebnisse einer bestimmten Stichprobenziehung zu wiederholen, wenn wir den Startwert für die Erzeugung der Zufallszahlen vorgeben

```
set.seed(1)
sample(ee, 3)
## [1] 2 7 4
set.seed(1)
sample(ee, 3)
## [1] 2 7 4
```